

Cell 9 — Tuning Logs + Final Comparison (Notebook cell index: 18)

Builds DataFrames from logs, displays ranked tables, plots macro-F1 bars, prints best/worst rows, saves CSV files.

Granular line-by-line breakdown

Lists → DataFrames

```
tuning_df = pd.DataFrame(tuning_logs)
results_df = pd.DataFrame(all_results)
```

- `tuning_logs` — List of dicts from `add_tuning_log` (each dict → one row).
 - `all_results` — List of dicts from `evaluate_and_store` (test-set summaries).
 - `pd.DataFrame(...)` — Converts list-of-dicts to a table with columns = dict keys.
-

Tuning preview

```
print('Tuning log rows:', len(tuning_df))
```

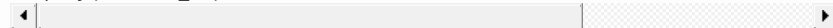
- `len(tuning_df)` — Number of validation experiments logged.

```
display(tuning_df.sort_values('val_macro_f1', ascending=False).head(20))
```

- `sort_values('val_macro_f1', ascending=False)` — Sort rows by validation macro-F1 **descending** (best first).
 - `.head(20)` — Top 20 rows only.
 - `display` — Jupyter rich display (HTML table).
-

Sort final test results

```
results_df = results_df.sort_values(['macro_f1', 'accuracy'], ascending=False)
display(results_df)
```



- `sort_values([...])` — Primary key `macro_f1`, tie-breaker `accuracy`; **both** descending.
 - `reset_index(drop=True)` — Replaces index with 0..N-1 so `iloc[0]` is truly rank 1.
-

Bar plot

```
plt.figure(figsize=(14, 5))
```

- Wide figure for many model names on x-axis.

```
sns.barplot(data=results_df, x='model', y='macro_f1', hue='preprocessing')
```

- `data=results_df` — Source table.
- `x='model'` — Categorical axis = model name column.
- `y='macro_f1'` — Bar height = test macro-F1.
- `hue='preprocessing'` — Splits each x tick into **grouped bars** for `none / extreme / optimum`.

```
plt.title('Macro F1 Comparison by Model and Preprocessing')
plt.xticks(rotation=35)
plt.tight_layout()
plt.show()
```

- `rotation=35` — Angles long model names for readability.
-

Best and worst experiments

```
best_exp = results_df.iloc[0]
worst_exp = results_df.iloc[-1]
```

- `iloc[0]` — **First row** after sorting = **best** macro-F1 (then accuracy).
- `iloc[-1]` — **Last row** = lowest ranked among recorded experiments.

```
print('Best-performing experiment:')
print(best_exp)

print('\nWorst-performing experiment:')
print(worst_exp)
```

- `print(best_exp)` — Prints **Series** one row (named index = column names).
-

Save CSVs

```
results_df.to_csv('final_results_table.csv', index=False)
tuning_df.to_csv('tuning_logs_table.csv', index=False)
```

- `to_csv` — Writes **DataFrame** to disk.
- `index=False` — Do not write row numbers as extra column (cleaner for Excel/report).

```
print('\nSaved: final_results_table.csv, tuning_logs_table.csv')
```

- Confirms filenames (saved in **current working directory** when notebook runs).
-

Mechanism of the whole cell

Materialize tables → sort for interpretation → visualize → highlight extremes → export artifacts.

Role and importance in the notebook

Delivers the **comparative summary** of all prior cells: one place to see which preprocessing × model × representation performed best and to archive numbers for the write-up.